Cloud Security & DevSecOps Mock Interview Notes

AWS • CSPM • EKS/Kubernetes • Terraform • CI/CD • Incident Response

Prepared from the mock-interview practice in this chat

How to use these notes

These notes reorganize the questions, answers, and follow-up discussions from the practice session into topic-based sections. The answers are polished interview responses rather than verbatim transcripts. Use them as speaking guides: understand the flow, then answer naturally in your own words.

- Recommended answer structure: define the concept → explain the AWS/Kubernetes control → give a practical example → explain monitoring or prevention.
- For scenario questions, use: Detect → Investigate → Contain/Remediate → Prevent → Monitor.
- Avoid claiming a tool does more than it actually does. For example, AWS Config detects configuration compliance; SCPs set permission guardrails; Security Hub aggregates and prioritizes findings.

Contents

- 1. Introduction and role alignment
- 2. AWS security foundations and multi-account governance
- 3. Centralized logging, CSPM and Security Hub
- 4. S3 security and automated remediation
- 5. IAM security and access management
- 6. AWS perimeter and application protection
- 7. Terraform and infrastructure security
- 8. AWS incident response and threat detection
- 9. Kubernetes and EKS security architecture
- 10. Kubernetes RBAC and network policies
- 11. Container and CI/CD security
- 12. Quick revision sheets and interview reminders

1. Introduction and role alignment

1.1 Tell me about yourself

Interviewer question: Please introduce yourself and summarize your experience relevant to a Cloud Security / DevSecOps Engineer role.

Suggested answer: Hi, I'm Varish Ansari. I have 4 years and 5 months of experience in Cloud Engineering, DevOps, and Cloud Security.

Currently, I'm working with Deloitte as an external consultant, supporting SAP's Cloud Security Posture Management (CSPM) initiatives.

Previously, I worked with a startup called SustainabilityEconomics.ai, where I managed AWS infrastructure, Kubernetes environments, and infrastructure automation using Terraform. I also worked on implementing AWS cloud security practices.

I have completed my M.Tech from NIT Nagpur. I also hold AWS Solutions Architect – Associate and HashiCorp Terraform Associate certifications.

Going forward, I'm looking for an opportunity where I can further strengthen my expertise in Cloud Security, CSPM, and DevSecOps, and contribute to building secure and scalable cloud environments.

Cross-question / follow-up: Why are you interested in this role? Emphasize ownership of cloud guardrails, security posture, automation, Kubernetes security and secure delivery pipelines.

Technical notes: Keep the introduction to roughly 60–90 seconds. Use consistent experience duration and certification names.

2. AWS security foundations and multi-account governance

2.1 Preventive vs. detective controls

Interviewer question: What is the difference between preventive and detective controls in AWS Organizations?

Suggested answer: Preventive controls stop unwanted or unauthorized actions before they happen. In AWS Organizations, Service Control Policies (SCPs) can set permission guardrails across accounts—for example, preventing member accounts from disabling required security services. Detective controls identify misconfigurations or suspicious activity after it occurs. AWS Config detects non-compliant resource configurations, GuardDuty detects potential threats, and CloudTrail records API activity. In simple terms, preventive controls block prohibited actions, while detective controls identify security issues.

Cross-question / follow-up: Is AWS Config preventive? No. AWS Config is primarily detective. SCPs do not grant permissions; they define the maximum available permissions for principals in affected accounts.

Technical notes: Useful distinction: CloudTrail is an audit log of API activity; GuardDuty analyzes supported signals for threats; Security Hub centralizes findings and posture.

2.2 AWS Shared Responsibility Model

Interviewer question: Explain the AWS Shared Responsibility Model with an EC2 and VPC example.

Suggested answer: The AWS Shared Responsibility Model defines security responsibilities between AWS and the customer. Security of the cloud is AWS's responsibility: AWS secures physical data centers, hardware, networking infrastructure and virtualization. Security in the cloud is the customer's responsibility: customers secure what they deploy and configure, including guest operating systems, patches, applications, IAM permissions, security groups, data and network configuration. For EC2, AWS manages the physical host and hypervisor, while the customer manages the guest operating system and application. For a VPC, AWS provides the networking service, while the customer designs subnets, route tables and network access controls.

Cross-question / follow-up: Who patches an EC2 operating system? The customer. Who secures the physical server? AWS.

Technical notes: Responsibility varies by service. Managed services such as RDS and Lambda shift some infrastructure and platform responsibilities to AWS, but customers still secure data, identities, application code and configuration.

2.3 Centralized security across multiple AWS accounts

Interviewer question: You have Development, Testing and Production accounts. How would you centrally manage security and enforce policies?

Suggested answer: I would use AWS Organizations to group and govern the accounts. I would designate a security account as the delegated administrator for services such as Security Hub and GuardDuty, and a separate log archive account for centralized log storage. I would enable organization-wide CloudTrail, AWS Config, GuardDuty and Security Hub. SCPs would enforce preventive guardrails, while Config and Security Hub would help identify non-compliant resources and aggregate findings. This gives the security team centralized visibility and consistent controls across accounts.

Cross-question / follow-up: What does delegated administrator mean? It allows a designated member account to centrally administer supported security services for the organization. It does not mean all logs automatically move into that account.

Technical notes: Use separate account purposes where appropriate: management, security/tooling, log archive, and workload accounts. Apply least privilege and protect the log archive.

2.4 Preventing a developer from disabling CloudTrail

Interviewer question: CloudTrail is disabled in one production account. How would you detect, prevent and automatically remediate it?

Suggested answer: I would use AWS Config to detect whether the required CloudTrail trail is enabled and configured correctly, and send the finding to Security Hub. For prevention, I would use an SCP to deny unauthorized actions that could stop or delete the

organization trail, with carefully designed exceptions for approved administration. For remediation, an EventBridge rule can match the Config compliance change or relevant finding and invoke Lambda or an approved automation workflow to alert the team and restore the required configuration. I would also verify that organization-level CloudTrail delivers logs to the protected central log archive.

Cross-question / follow-up: How does EventBridge trigger Lambda? A service emits an event—for example, a Config compliance change. An EventBridge rule matches the event pattern and invokes the target Lambda function.

Technical notes: Do not rely only on auto-remediation. Preventive SCP guardrails and organization trails are stronger controls. Test automation carefully and avoid creating duplicate or conflicting trails.

2.5 Centralized logging: CloudTrail, VPC Flow Logs and GuardDuty

Interviewer question: How would you export logs and security findings from multiple accounts into a central location?

Suggested answer: For CloudTrail, I would create an organization trail and configure delivery to a central S3 bucket in the log archive account, protected by a restrictive bucket policy and encryption. For VPC Flow Logs, I would enable flow logs on the required VPCs, subnets or network interfaces and deliver them to a centralized S3 bucket or CloudWatch Logs destination, depending on the architecture. For GuardDuty and Security Hub, I would use the delegated administrator model to aggregate findings centrally. I would then integrate the relevant logs and findings with the SIEM/SOC for monitoring and response.

Cross-question / follow-up: Do we export GuardDuty logs in the same way as CloudTrail? Not necessarily. GuardDuty produces findings; use delegated administration and supported integrations or export mechanisms to centralize them. Security Hub aggregates findings rather than serving as the raw log archive.

Technical notes: CloudTrail records API activity. VPC Flow Logs record network flow metadata, not packet contents. Central log storage should have restricted access, encryption, retention controls and integrity protections.

2.6 What does Security Hub provide?

Interviewer question: What does AWS Security Hub provide that separate security alerts do not?

Suggested answer: AWS Security Hub provides a centralized view of security findings and security posture across AWS accounts and Regions. It can aggregate findings from services such as GuardDuty, Inspector, Macie and AWS Config, and evaluate the environment against supported security standards such as CIS AWS Foundations Benchmark and AWS Foundational Security Best Practices. It helps prioritize findings by severity and supports automated response workflows through EventBridge and services such as Lambda.

Cross-question / follow-up: Does Security Hub replace GuardDuty or Config? No. GuardDuty detects threats, Config evaluates resource configuration and compliance, while Security Hub centralizes and prioritizes findings.

Technical notes: Use delegated administrator and organization integration for multi-account visibility. Security Hub standards and controls may vary by enabled standard and service configuration.

3. Centralized logging, CSPM and Security Hub

3.1 Security Hub finding: public S3 bucket in production

Interviewer question: Security Hub reports that an S3 bucket in a production account is publicly accessible. How do you investigate, remediate and prevent recurrence?

Suggested answer: First, I would confirm the finding and its severity using Security Hub and AWS Config. Second, I would investigate the bucket policy, ACLs, Block Public Access settings and business requirement to determine whether public access is intentional. If it is unauthorized, I would remove the unintended public permissions and enable the appropriate S3 Block Public Access settings. For automation, an EventBridge rule can match the finding and trigger a Lambda remediation workflow, subject to safeguards and approval requirements. To prevent recurrence, I would use organizational guardrails where appropriate, Terraform security scanning in CI/CD, and continuous AWS Config monitoring.

Cross-question / follow-up: What if the bucket intentionally hosts public images or PDFs? Validate the use case first. Do not blindly remove required public access; consider CloudFront with Origin Access Control for private S3 origins where appropriate.

Technical notes: Example of a risky policy: Principal set to '*' with s3:GetObject on all objects. Check bucket-level and account-level Block Public Access, bucket policies, ACLs and access points. Test remediation before enabling automatic changes in production.

3.2 Security Hub severity and response

Interviewer question: How would you prioritize Security Hub findings?

Suggested answer: I would consider severity, exploitability, exposure, business criticality, data sensitivity and whether the resource is production-facing. Critical or high-risk findings on sensitive production resources receive urgent triage. I would assign ownership, investigate the root cause, remediate or formally accept the risk with an approved exception, and track closure.

Cross-question / follow-up: Does a finding marked Critical always mean immediate automatic deletion or blocking? No. Severity guides prioritization; remediation should consider context and business impact.

4. IAM security and access management

4.1 Replace excessive AdministratorAccess

Interviewer question: An IAM user has AdministratorAccess but only needs S3 and EC2. How would you reduce access safely?

Suggested answer: I would first review the user's actual access requirements and activity using IAM credential reports, service last-accessed information and IAM Access Analyzer capabilities where relevant. I would identify the exact S3 buckets and EC2 actions required, then replace AdministratorAccess with a least-privilege policy. I would test the policy with the user or application, monitor for denied actions and refine it. Permission boundaries can be used as an additional guardrail to limit the maximum permissions that can be granted.

Cross-question / follow-up: Can a permission boundary alone remove existing excessive permissions? No. It limits the maximum permissions; the excessive identity policy must still be corrected.

Technical notes: IAM Access Analyzer helps identify external access and validate policies. For unused credentials, use credential reports and access-key last-used information—not Access Analyzer as the primary unused-key report.

4.2 Unused IAM access key

Interviewer question: An IAM access key has not been used for 120 days but is still active. What would you do?

Suggested answer: I would identify the key using the IAM credential report and access-key last-used information. I would confirm with the application owner that no workload depends on it. I would deactivate the key first, monitor for any impact, and delete it after confirming it is no longer required. Longer term, I would prefer IAM roles and temporary credentials over long-lived access keys.

Cross-question / follow-up: Should you immediately delete an unused key? Safer practice is to deactivate, monitor, then delete after confirming no dependency.

4.3 Break-glass production access

Interviewer question: An engineer needs temporary administrator access during a critical production incident. How would you design break-glass access?

Suggested answer: I would create a pre-approved, tightly controlled break-glass role rather than creating an ad hoc administrator user during the incident. Access should require strong MFA, an emergency approval or documented incident process, and be time-bound through an appropriate identity and access workflow. All actions must be logged in CloudTrail and monitored. Access should be revoked or allowed to expire immediately after the incident, followed by a review of actions performed.

Cross-question / follow-up: What should happen after the incident? Revoke access, review CloudTrail, document the reason and actions, and rotate credentials if exposure is suspected.

Technical notes: Use temporary role sessions where possible. Avoid sharing permanent administrator credentials.

4.4 EKS workload needs S3 access: IAM user or IRSA?

Interviewer question: A developer asks you to create an IAM user, generate an access key and store it in a Kubernetes Secret so an EKS application can access S3. Would you approve?

Suggested answer: I would not approve long-lived IAM user access keys stored in Kubernetes Secrets for this use case. I would create a least-privilege IAM policy for the required S3 bucket actions, attach it to an IAM role, and associate that role with the pod's Kubernetes service account using EKS Pod Identity or IAM Roles for Service Accounts (IRSA). The pod then obtains temporary AWS credentials through the supported mechanism and accesses S3 without hardcoded access keys.

Cross-question / follow-up: Do we need Secrets Manager for the S3 access key? No. With IRSA or EKS Pod Identity, the workload gets temporary AWS credentials. Secrets Manager is appropriate for application secrets such as database passwords or API tokens.

Technical notes: Use a narrowly scoped S3 policy. For IRSA, configure the cluster OIDC provider and service-account role trust relationship. For EKS Pod Identity, configure the required association and agent/add-on support.

4.5 Secrets management for Lambda and Kubernetes

Interviewer question: How would you secure secrets for serverless or Kubernetes workloads?

Suggested answer: I would store application secrets such as database passwords in AWS Secrets Manager, encrypted with KMS. I would grant the workload a least-privilege IAM role to retrieve only the required secret at runtime. For EKS, I could use External Secrets Operator or another approved integration to retrieve secrets from Secrets Manager, with workload identity through IRSA or EKS Pod Identity. For CI/CD, I would use the platform's secret store and ensure secrets are never hardcoded in source code, Dockerfiles, manifests or logs. I would enable rotation where supported and monitor access.

Cross-question / follow-up: Should AWS access keys for S3 be stored in Secrets Manager when using EKS? No. Use workload identity and temporary credentials. Store application credentials, not AWS service access keys, where possible.

Technical notes: Kubernetes Secrets are base64-encoded by default, not automatically encrypted by that encoding. Configure encryption at rest and tightly restrict RBAC if Kubernetes Secrets are used.

5. AWS perimeter and application protection

5.1 Protect a public application behind CloudFront and ALB

Interviewer question: How would you secure a public-facing application behind CloudFront and an Application Load Balancer?

Suggested answer: I would place CloudFront in front of the ALB and associate AWS WAF with the CloudFront distribution. I would use AWS Managed Rule Groups for common web attacks such as SQL injection and cross-site scripting, plus custom rules for rate limiting, IP restrictions, geo restrictions and bot control where needed. I would also prevent users from bypassing CloudFront and reaching the ALB directly, for example by restricting ALB ingress to CloudFront origin-facing traffic using an appropriate origin access control pattern such as a CloudFront managed prefix list and additional verification. Finally, I would enable WAF and CloudFront logging and send security events to central monitoring.

Cross-question / follow-up: Can WAF attach to CloudFront or ALB? AWS WAF can protect supported CloudFront distributions and regional resources such as ALBs. For this design, placing WAF at CloudFront provides edge filtering before traffic reaches the ALB.

Technical notes: Do not expose the ALB directly if the design requires all traffic to pass through CloudFront. Validate the origin restriction carefully; a prefix-list restriction alone may not always prove the request came from your distribution.

5.2 Rate limiting and bot mitigation

Interviewer question: An application receives excessive requests from a small set of IP addresses. How would you detect and mitigate the traffic?

Suggested answer: For basic mitigation, I would configure an AWS WAF rate-based rule with a suitable request threshold and evaluation window. WAF can automatically block or challenge traffic that exceeds the threshold. I would enable WAF logging and monitor the source IPs and request patterns. For advanced response, WAF logs or findings can be sent through a processing pipeline;

EventBridge can match a relevant event and invoke Lambda to update a WAF IP set after validation. I would set thresholds carefully to avoid blocking legitimate users.

Cross-question / follow-up: Is EventBridge plus Lambda alone enough? No. An event source must emit an event, such as a CloudWatch alarm, a finding or a log-processing workflow. EventBridge matches it and invokes Lambda. Basic WAF rate limiting does not need Lambda.

Technical notes: Rate-based rules are the first line of defense. Avoid relying on delayed log analysis for immediate rate limiting.

6. Terraform and infrastructure security

6.1 Secure Terraform state in S3

Interviewer question: How do you secure Terraform state when using S3 as the backend?

Suggested answer: I would store the state in a dedicated S3 bucket with server-side encryption using KMS, enable versioning, and restrict access through least-privilege IAM policies and bucket policies. I would enable appropriate logging and monitoring, such as CloudTrail data events where required. For state locking, I would use the locking mechanism supported by the Terraform version and backend configuration in use. Historically, DynamoDB was commonly used for S3 backend locking; newer Terraform versions support S3-native lock files, so I would follow the team's supported version and current backend guidance.

Cross-question / follow-up: Why protect the state file? It may contain sensitive infrastructure metadata and sometimes secret values. Treat it as sensitive data.

Technical notes: Do not assume DynamoDB locking is mandatory for every current Terraform S3 backend. Confirm the Terraform version and configured locking approach.

6.2 Detect, prevent and remediate SSH open to the world

Interviewer question: Several EC2 security groups allow SSH (port 22) from 0.0.0.0/0. How would you detect, prevent and fix this?

Suggested answer: Before deployment, I would scan Terraform or IaC files in CI/CD using tfsec, Checkov or Trivy configuration scanning, and fail the pipeline when an insecure SSH rule is found. After deployment, AWS Config can detect security groups that allow port 22 from 0.0.0.0/0 and mark them non-compliant. EventBridge can match the compliance event and invoke Lambda or an approved remediation workflow to remove the specific insecure ingress rule—not delete the entire security group. I would notify the owner and verify compliance afterward.

Cross-question / follow-up: What is the difference between tfsec/Trivy and Config? IaC scanners find issues before deployment; Config detects deployed resource configuration after deployment.

Technical notes: Prefer Systems Manager Session Manager or restricted administrative access instead of public SSH where practical.

7. AWS incident response and threat detection

7.1 GuardDuty detects suspicious activity from an EC2 instance role

Interviewer question: GuardDuty reports suspicious activity associated with an EC2 instance role. What are your immediate actions?

Suggested answer: My first priority is containment. I would investigate the GuardDuty finding, identify the affected instance and role, and review CloudTrail activity. I would isolate the EC2 instance using a restricted security group or other containment method, and restrict the compromised role's permissions or sessions using an incident response procedure. I would preserve evidence, including EBS snapshots and relevant logs, then investigate the root cause. After removing malware or unauthorized changes and patching the weakness, I would rebuild from a trusted image where appropriate, restore the application securely and monitor for recurrence.

Cross-question / follow-up: Should you immediately stop or terminate the EC2 instance? Not automatically. Isolate it and preserve evidence first, following the incident response plan. Stopping can affect volatile evidence and business operations.

Technical notes: Detaching a role does not necessarily invalidate credentials already issued. Restrict permissions and address active sessions as part of containment.

8. Kubernetes and EKS security architecture

8.1 Secure architecture for an application on EKS

Interviewer question: What security controls would you implement when designing a Kubernetes application platform on AWS?

Suggested answer: I would design the VPC across multiple Availability Zones with public subnets for internet-facing entry points and private subnets for EKS worker nodes and application workloads. I would place CloudFront and AWS WAF in front of the public load balancer, and keep backend and database tiers private. I would use VPC endpoints for supported AWS services such as S3 and DynamoDB where appropriate. Inside EKS, I would enforce RBAC, least-privilege service accounts, network policies, non-root containers, restricted pod security settings, image scanning and secure secrets management. For AWS service access from pods, I would use IRSA or EKS Pod Identity. I would enable CloudTrail, GuardDuty, Security Hub and relevant logging for monitoring and compliance.

Cross-question / follow-up: How does a pod access S3 securely? Use a least-privilege IAM role associated with the Kubernetes service account through IRSA or EKS Pod Identity; do not place long-lived AWS keys in pod Secrets.

Technical notes: Use a supported network policy implementation. EKS VPC CNI Network Policy support or Calico may be options depending on cluster configuration. Multi-region disaster recovery is a separate availability design decision and should be based on RTO/RPO.

8.2 Kubernetes RBAC

Interviewer question: What is Kubernetes RBAC, and what is the difference between Role, ClusterRole, RoleBinding and ClusterRoleBinding?

Suggested answer: RBAC stands for Role-Based Access Control. It is a Kubernetes authorization mechanism that controls what authenticated users, groups and service accounts can do. A Role defines permissions within a namespace. A ClusterRole defines cluster-scoped permissions or reusable permissions that can also be bound within a namespace. A RoleBinding grants a Role or ClusterRole to a subject within a namespace. A ClusterRoleBinding grants a ClusterRole across the cluster. For example, if a developer needs read-only access to pods in the development namespace, I would create a Role with get, list and watch permissions and bind it to the developer using a RoleBinding.

Cross-question / follow-up: Authentication identifies the user; authorization determines what the user can do. In EKS, IAM can authenticate identities, while Kubernetes RBAC authorizes access to Kubernetes API resources.

Technical notes: RoleBinding can bind a ClusterRole but the resulting permissions are scoped to the RoleBinding's namespace. ClusterRoleBinding grants the permissions cluster-wide.

8.3 Restrict pod-to-pod and namespace communication

Interviewer question: How would you restrict communication between pods and namespaces?

Suggested answer: I would use Kubernetes Network Policies to control pod network traffic. Ingress rules control traffic entering selected pods, while egress rules control traffic leaving them. For example, I could allow frontend pods to connect to backend pods on TCP 8080 and backend pods to connect to database pods on TCP 5432, while denying other unnecessary traffic. I would use namespace and pod selectors to isolate Development, Testing and Production workloads. In Amazon EKS, I would ensure the chosen CNI or network plugin supports and enforces Network Policies.

Cross-question / follow-up: Does Kubernetes block all pod-to-pod traffic by default? In a typical cluster, pod communication is generally allowed unless policies restrict it. Network Policies are additive and require a compatible enforcement implementation.

Technical notes: Network Policy is not a substitute for IAM or security groups. Use each control at the appropriate layer.

8.4 Container running as root or privileged

Interviewer question: A developer deploys a container running as root with privileged access in production. How would you prevent this?

Suggested answer: I would first identify the affected pod and inspect its securityContext. I would enforce runAsNonRoot: true, set allowPrivilegeEscalation: false and privileged: false, drop unnecessary Linux capabilities, and use a read-only root filesystem where compatible. I would apply the Restricted Pod Security Standard and use admission controllers such as Kyverno or OPA Gatekeeper to validate manifests and block non-compliant workloads before they are admitted. For an already-running insecure pod, I would contain the risk, investigate and redeploy a corrected workload.

Cross-question / follow-up: What is the difference between CI scanning and admission control? CI scanning checks manifests before deployment; admission control enforces policy at the Kubernetes API server.

Technical notes: Ensure the container image supports the selected non-root UID and read-only filesystem.

8.5 Prevent unauthorized cluster-admin RoleBindings

Interviewer question: A developer accidentally adds a ClusterRoleBinding that grants cluster-admin to their own service account. How would you prevent this in CI/CD?

Suggested answer: I would enforce least-privilege RBAC and restrict who can create or modify cluster-wide role bindings. I would scan Kubernetes manifests in CI using tools such as Kubesec or Checkov for risky RBAC configurations. I would also use an admission controller such as OPA Gatekeeper or Kyverno to reject unauthorized bindings to cluster-admin. Production changes should require appropriate review and approval.

Cross-question / follow-up: What does the admission controller add? It enforces policy at the API server, so an unsafe manifest is rejected even if it bypasses the normal CI pipeline.

Technical notes: Scanning tool capabilities vary; use explicit policy rules to detect privileged ClusterRoleBindings rather than assuming every scanner catches every case.

9. Container and CI/CD security

9.1 Scan container images in a secure CI/CD pipeline

Interviewer question: How would you ensure vulnerable or malicious Docker images do not reach production EKS?

Suggested answer: After building the image, I would scan it using Trivy or Amazon Inspector for operating-system package and application dependency vulnerabilities. I would configure security gates so builds fail when vulnerabilities violate our severity policy—for example, blocking critical findings and requiring review or remediation for high findings. Only approved images would be pushed to Amazon ECR and deployed to EKS. I would also monitor images after deployment because new vulnerabilities may be disclosed later.

Cross-question / follow-up: What if Trivy finds five critical and ten high vulnerabilities? Block the release under the defined policy, review findings, update vulnerable packages or base images, rebuild and rescan. Do not waive findings without documented risk acceptance.

Technical notes: Use immutable image tags or digests, trusted base images, image provenance/signing where available, and ECR scanning or Inspector integration.

9.2 A new CVE is announced after the image scan passed

Interviewer question: What if a new CVE is announced the day after the CI scan passed?

Suggested answer: I would update the scanner vulnerability database and rescan the affected image versions already in use. If the new CVE affects our image, I would assess severity and exploitability, prioritize based on production exposure, patch the vulnerable package or base image, rebuild and rescan. For a critical actively exploitable issue, I would consider temporary containment, disabling the vulnerable feature or rolling back while a fixed image is prepared. I would then redeploy the approved image and track the finding to closure.

Cross-question / follow-up: Why scan again after deployment? Vulnerability databases change, so an image that passed yesterday can become vulnerable when a new CVE is published.

9.3 Prevent hardcoded secrets in Dockerfiles and Kubernetes YAML

Interviewer question: How would you prevent developers from hardcoding AWS keys or database passwords in source code, Dockerfiles or Kubernetes manifests?

Suggested answer: I would integrate secret scanning tools such as GitHub Secret Scanning, Gitleaks or GitGuardian into pull requests and CI/CD. If a secret is detected, the pipeline should fail and the secret should be treated as exposed. Application secrets such as database passwords should be stored in AWS Secrets Manager, encrypted with KMS, and retrieved at runtime using a least-privilege workload role. For EKS, I would use IRSA or EKS Pod Identity for AWS service access and an approved Secrets Manager integration for application secrets. I would also prevent secrets from appearing in logs or build artifacts.

Cross-question / follow-up: Should AWS S3 access keys be stored in Secrets Manager for an EKS pod? Prefer workload identity with temporary credentials. Secrets Manager is suitable for application secrets such as database credentials.

9.4 AWS access key accidentally committed to GitHub

Interviewer question: A developer commits an AWS access key to GitHub and secret scanning detects it. What is your response?

Suggested answer: My immediate priority is containment: deactivate the exposed key. I would then investigate CloudTrail to identify any suspicious API calls and assess what resources may have been accessed or changed. I would remove the secret from the repository and address exposure in Git history, while remembering that deleting the commit does not make the credential safe. If the application still requires credentials, I would replace them safely and verify dependent services. Finally, I would integrate secret scanning into pull requests and CI/CD and prefer IAM roles or temporary credentials over long-lived keys.

Cross-question / follow-up: Why deactivate before creating a replacement? The exposed credential may already be compromised. Contain first, then rotate safely and verify application dependencies.

Technical notes: Also review permissions of the exposed key, related secrets and any resources accessed during the exposure window.

9.5 Secure CI/CD pipeline for microservices deployed to EKS

Interviewer question: Where would you integrate security scanning in a secure CI/CD pipeline?

Suggested answer: I would integrate security checks at multiple stages. For source code, I would use SonarQube or Semgrep for static analysis. For secrets, I would use GitHub Secret Scanning or Gitleaks. For dependencies, I would use Trivy or Snyk. For Terraform, I would use tfsec, Checkov or Trivy configuration scanning. After building the container image, I would use Trivy or Amazon Inspector to scan it. I would configure security gates so the pipeline fails when findings violate policy. After approval, I would push the image to ECR and deploy it to EKS. At runtime, I would use Kubernetes security controls, GuardDuty, Security Hub and CloudWatch for monitoring.

Cross-question / follow-up: Can scans run in parallel? Independent checks can run in parallel to reduce pipeline time, while dependent stages—such as image scanning after image build—must follow the required order.

Technical notes: Security Hub is for centralized findings and posture management; GuardDuty detects threats. Neither replaces source, dependency, IaC or image scanning.

10. Quick revision sheets and interview reminders

10.1 Tool-to-purpose cheat sheet

Tool / Service	Primary purpose
AWS Organizations / SCP	Multi-account governance and preventive permission guardrails
AWS Config	Resource configuration compliance and drift detection
CloudTrail	AWS API activity audit trail
GuardDuty	Threat detection from supported telemetry
Security Hub	Centralized security findings and posture management
IAM Access Analyzer	Analyze resource access and validate policies
Secrets Manager	Store and rotate application secrets
KMS	Create and manage encryption keys
AWS WAF	Layer 7 web request filtering and rate-based rules
CloudFront	Content delivery and edge entry point
EventBridge	Event routing and automation triggers
Lambda	Run event-driven remediation logic
Trivy	Container, dependency and IaC scanning
SonarQube / Semgrep	Static application code analysis
Gitleaks / GitHub Secret Scanning	Detect exposed secrets
tfsec / Checkov	Terraform and IaC security checks
IRSA / EKS Pod Identity	Give EKS workloads temporary AWS credentials
Kubernetes RBAC	Authorize users, groups and service accounts
NetworkPolicy	Control pod ingress and egress traffic
Kyverno / OPA Gatekeeper	Enforce Kubernetes admission policies
Pod Security Standards	Baseline security requirements for pods
Amazon Inspector	Vulnerability management for supported workloads and images

10.2 Common interview corrections

- AWS Config is detective, not a preventive control. Use SCPs and preventive policy enforcement for guardrails.
- SCPs do not grant permissions; they set permission boundaries for principals in member accounts.
- Security Hub centralizes findings; GuardDuty detects threats; Config evaluates resource configuration; CloudTrail records API activity.
- For EKS-to-S3 access, prefer IRSA or EKS Pod Identity over IAM user access keys in Kubernetes Secrets.
- For unused IAM keys, use credential reports and last-used information; deactivate first, monitor, then delete.
- For open SSH, remove the specific 0.0.0.0/0 port 22 rule—not the entire security group.
- For a compromised EC2 instance, contain and preserve evidence before destructive actions.
- WAF rate-based rules can mitigate excessive requests directly; EventBridge/Lambda is for additional event-driven response.
- Network Policies require a CNI/network plugin that supports enforcement.
- A scan that passed previously does not guarantee the image remains vulnerability-free; rescan when new CVEs are disclosed.

10.3 Scenario answer framework

For most Cloud Security and DevSecOps scenario questions, organize the answer in this order:

1. Detect the issue and identify affected accounts/resources.
2. Investigate the root cause, business impact and scope.
3. Contain immediate risk without destroying evidence.
4. Remediate the specific misconfiguration or vulnerability.
5. Prevent recurrence using guardrails, CI/CD policy checks and least privilege.
6. Monitor, validate compliance and document closure.

10.4 Suggested next practice topics

- EKS authentication: IAM, access entries, OIDC, IRSA and EKS Pod Identity.
- Kubernetes Secrets encryption at rest and External Secrets Operator.
- Admission control implementation using Kyverno or OPA Gatekeeper.
- EKS control plane logs, audit logging and CloudWatch integration.
- Terraform policy-as-code and drift detection.
- SOC/SIEM integration: GuardDuty and Security Hub findings to a SIEM.
- AWS WAF managed rule groups, rate-based rules and CloudFront origin protection.
- Compliance evidence for CIS, ISO 27001, PCI DSS and applicable regulatory requirements.

End of notes